

Chapter 5

Parsers: Top-Down Parsing, FIRST & FOLLOW, LL(1)

From tokens to a parse tree: what a parser decides, how recursive descent and LL(1) parsers decide it, and how to compute the sets that make the decision

Compiler Construction Module

Based on the lecture “Parsers” (the slides are titled Lecture 4), expanded

How to read this chapter

The lexer (Chapters 2–4) turns characters into tokens. The **parser** takes those tokens and checks that they form a sentence of the language, building a *parse tree* that shows how. This chapter covers the top-down half of parsing, which is what the lecture develops in detail:

- what a parser decides, and how top-down and bottom-up parsers differ (§2);
- how to prepare a grammar for top-down parsing: removing left recursion and left factoring (§3). The slides state the requirement but not the method;
- recursive descent parsers, with the slides’ code run and debugged (§4);
- FIRST and FOLLOW sets: definitions, rules, the iterative algorithm, and every slide exercise worked out (§6, §7);
- the LL(1) parsing table, the stack algorithm, and how to decide whether a grammar is LL(1) (§8, §9).

Every FIRST set, FOLLOW set, table and trace in this chapter was checked by a program, and the slides’ parser was compiled and run. Section 10 ends with a map from the *FIRST & FOLLOW Advanced Practice Sheet* to the sections that prepare each exercise.

Slides	Topic	Section
1–5	parser, parse tree, top-down vs bottom-up, classification	§2
6	requirements: no left recursion, no non-determinism	§3
7–16	recursive descent: code, trace, call stack	§4
17–20	LL(1) parser, FIRST, FOLLOW definitions	§6, §7
21–37	derivation of FIRST and FOLLOW, exercises	§6–§7
38–45	LL(1) table construction, LL(1) parsing	§8
46–66	“is this grammar LL(1)?”, homework	§9

1 Grammars in One Page

The slides assume context-free grammars without recalling them. Everything below rests on these definitions.

Concept: Context-free grammar

A **context-free grammar** (CFG) is $G = (N, T, P, S)$:

- N : **non-terminals**, written in upper case (S, A, E, E'). They name syntactic categories: “expression”, “statement”.
- T : **terminals**, written in lower case or as symbols ($a, \text{id}, +, ()$). These are the *tokens* produced by the lexer.
- P : **productions** $A \rightarrow \alpha$, where α is a string of terminals and non-terminals, possibly empty ($A \rightarrow \varepsilon$). $A \rightarrow \alpha \mid \beta$ abbreviates two productions.
- S : the **start symbol**, by convention the left side of the first production.

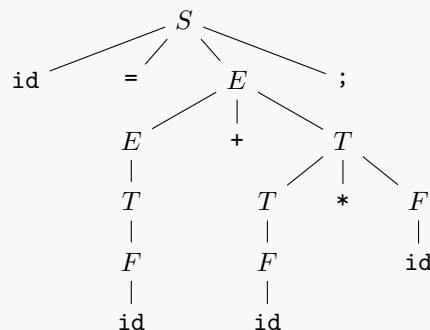
Greek letters α, β, γ stand for strings of grammar symbols; ε is the empty string; $\$$ marks the end of the input.

Derivations. One step $\alpha A \gamma \Rightarrow \alpha \beta \gamma$ replaces a non-terminal A by the right side of one of its productions $A \rightarrow \beta$. $\Rightarrow^*$ means zero or more steps. The **language** of G is the set of terminal strings w with $S \Rightarrow^* w$. In a **leftmost** derivation the leftmost non-terminal is always the one replaced; in a **rightmost** derivation, the rightmost one.

Parse tree. The root is S ; each internal node A has as children the symbols of the production used to expand it; the leaves, read left to right, spell the input. Every parse tree corresponds to exactly one leftmost derivation and one rightmost derivation. A grammar is **ambiguous** if some string has two different parse trees (equivalently, two leftmost derivations); no deterministic parser of the kind studied here can handle an ambiguous grammar without an extra rule.

Worked example: The slides’ first parse tree

Grammar: $S \rightarrow \text{id} = E ;$, $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow \text{id}$. The token string $\text{id} = \text{id} + \text{id} * \text{id} ;$ has the parse tree



The tree shows the structure the later phases need: $*$ is grouped below $+$, so $\text{id} + \text{id} * \text{id}$ means $\text{id} + (\text{id} * \text{id})$. The grammar encodes precedence by putting $*$ one level deeper.

2 What a Parser Decides

A **parser** is a program that, given a token string, builds a parse tree for it if the string is generated by the grammar, and reports an error otherwise. There are two ways to build the tree.

Worked example: Top-down versus bottom-up on aabcde

Grammar: $S \rightarrow aABe$, $A \rightarrow Abc \mid a$, $B \rightarrow d$.

Top-down: start at S , expand toward the leaves

$S \Rightarrow aABe \Rightarrow aAbcBe \Rightarrow aabcBe \Rightarrow aabcde$

a **leftmost derivation**, built forwards

Decision at each step: **which production** to use for the leftmost non-terminal

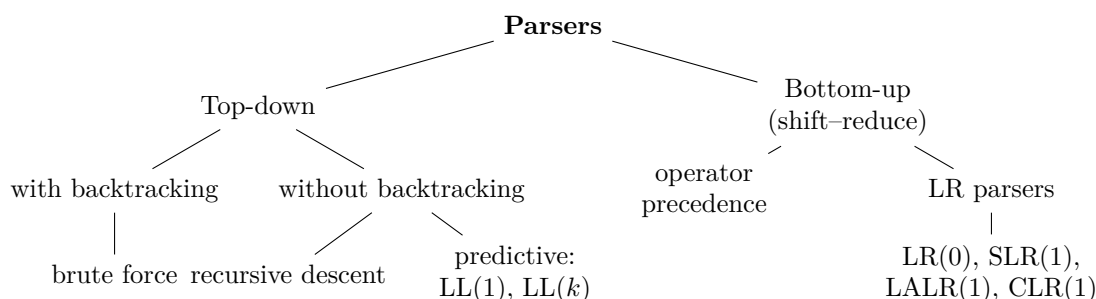
Bottom-up: start at the leaves, reduce toward S

$aabcde \leftarrow aAbcde \leftarrow aAde \leftarrow aABe \leftarrow S$

a **rightmost derivation**, built in reverse

Decision at each step: **when to reduce**, and by which production

Read the bottom-up column right to left: $S \Rightarrow aABe \Rightarrow aAde \Rightarrow aAbcde \Rightarrow aabcde$ is a rightmost derivation (the rightmost non-terminal is expanded each time). A bottom-up parser discovers it backwards, by finding a right side in the current string and replacing it by its left side.



The lecture develops the top-down branch. Bottom-up parsers are more powerful (they accept left-recursive grammars, for instance) and are the subject of a later lecture.

3 Preparing a Grammar for Top-Down Parsing

The slides state that a grammar used for top-down parsing should have neither **left recursion** nor **non-determinism**. This section explains why, and how to remove both.

3.1 Left recursion

A grammar is **left-recursive** if some non-terminal A can derive a string starting with itself: $A \Rightarrow^+ A\alpha$. It is *directly* left-recursive if a production has the form $A \rightarrow A\alpha$, and *indirectly* left-recursive otherwise (for example $S \rightarrow Aa$, $A \rightarrow Sd$, so $S \Rightarrow Aa \Rightarrow Sda$).

Why it breaks top-down parsing. A top-down parser for $A \rightarrow A\alpha \mid \beta$ that chooses $A \rightarrow A\alpha$ must, as its first step, parse an A again, at the same input position, without consuming anything. In a recursive descent parser this is the procedure $A()$ calling $A()$ immediately: infinite recursion. In table form it shows up as a conflict, as the following proposition proves.

Proposition 3.1 (Left recursion always produces a FIRST/FIRST clash). *If $A \rightarrow A\alpha \mid \beta$ and $\text{FIRST}(\beta)$ contains a terminal a , then $a \in \text{FIRST}(A\alpha)$ too. So both productions want the same table cell, and the grammar is not $LL(1)$.*

Proof. $\beta \Rightarrow^* a\gamma$ for some γ , so $A\alpha \Rightarrow \beta\alpha \Rightarrow^* a\gamma\alpha$. Hence $a \in \text{FIRST}(A\alpha)$. (FIRST is defined in §6; the claim only uses “the first terminal of something derivable”). $\square$

Method: Removing direct left recursion

Group the productions of A into those that start with A and those that do not:

$$A \rightarrow A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_n \quad (\text{no } \beta_j \text{ starts with } A).$$

Replace them by, with a new non-terminal A' :

$$A \rightarrow \beta_1 A' \mid \cdots \mid \beta_n A', \quad A' \rightarrow \alpha_1 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon.$$

Why it is correct. Both versions generate exactly the strings $\beta_j \alpha_{i_1} \alpha_{i_2} \cdots \alpha_{i_k}$ for $k \geq 0$: one β followed by any sequence of α 's. The left-recursive version builds this sequence right to left ($A \Rightarrow A\alpha \Rightarrow A\alpha\alpha \Rightarrow \cdots \Rightarrow \beta\alpha \cdots \alpha$); the new version builds it left to right ($A \Rightarrow \beta A' \Rightarrow \beta\alpha A' \Rightarrow \cdots$). The language is unchanged; only the shape of the parse tree changes.

Worked example: The slides' expression grammar comes from exactly this

$E \rightarrow E+T \mid T$ has $\alpha = +T$ and $\beta = T$, so it becomes

$$E \rightarrow TE', \quad E' \rightarrow +TE' \mid \varepsilon.$$

Likewise $T \rightarrow T*F \mid F$ becomes $T \rightarrow FT'$, $T' \rightarrow *FT' \mid \varepsilon$. This is precisely the grammar the slides use for FIRST and FOLLOW (§6). The left-recursive version, checked by program, has clashes on `id` and `(` for both E and T ; the transformed one is LL(1). The two generate the same strings (checked on all strings up to 7 tokens).

Method: Removing indirect left recursion

Order the non-terminals $A_1, \dots, A_n$. For $i = 1, \dots, n$:

1. for each $j < i$, replace every production $A_i \rightarrow A_j \gamma$ by $A_i \rightarrow \delta_1 \gamma \mid \cdots \mid \delta_k \gamma$, where $A_j \rightarrow \delta_1 \mid \cdots \mid \delta_k$ are the *current* productions of A_j ;
2. remove the direct left recursion of A_i with the method above.

After step i , no production of A_i starts with $A_1, \dots, A_i$, so no cycle back to the start of a production can remain. The algorithm assumes no cycles $A \Rightarrow^+ A$ and handles ε -productions with care: if some A_j is nullable, a production $A_i \rightarrow A_j A_k \gamma$ may hide left recursion through A_k .

Worked example: Indirect left recursion, step by step

$S \rightarrow Ab \mid c$, $A \rightarrow Sa \mid d$. Here $S \Rightarrow Ab \Rightarrow Sab$. Order S, A .

- $i = 1$ (S): no production of S starts with S . Nothing to do.
- $i = 2$ (A): substitute S in $A \rightarrow Sa$, giving $A \rightarrow Aba \mid ca \mid d$. Now remove the direct recursion ($\alpha = ba$, $\beta_1 = ca$, $\beta_2 = d$):

$$A \rightarrow ca A' \mid d A', \quad A' \rightarrow ba A' \mid \varepsilon.$$

The result $S \rightarrow Ab \mid c$, $A \rightarrow ca A' \mid d A'$, $A' \rightarrow ba A' \mid \varepsilon$ generates the same strings (c , db , cab , $dbab$, $\dots$; checked up to length 7). **But it is still not LL(1):** $S \rightarrow Ab$ and $S \rightarrow c$ both start with c (because $A \Rightarrow ca A'$), and A' has a FIRST/FOLLOW clash on b . Removing left recursion is *necessary* for top-down parsing, not *sufficient*.

3.2 Non-determinism and left factoring

“Non-determinism” on the slides means that two alternatives of a non-terminal start the same way, so one lookahead token cannot choose between them. In $A \rightarrow ab \mid a$, seeing **a** does not say which production to use.

Method: Left factoring

If $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \cdots \mid \alpha\beta_n \mid \gamma$ with a common prefix $\alpha \neq \varepsilon$, replace by

$$A \rightarrow \alpha A' \mid \gamma, \quad A' \rightarrow \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

(some β_i may be ε). Repeat until no two alternatives share a prefix. The parser now reads α once and postpones the choice until after it, when the next token can decide.

Worked example: Two left factorings

- $S \rightarrow cAd$, $A \rightarrow ab \mid a$ becomes $A \rightarrow aA'$, $A' \rightarrow b \mid \varepsilon$. Now $\text{FIRST}(b) = \{b\}$ and $\text{FOLLOW}(A') = \{d\}$ are disjoint: LL(1) (checked).
- The **dangling else**: $S \rightarrow iEtS \mid iEtSeS \mid a$, $E \rightarrow b$ ($i = \text{if}$, $t = \text{then}$, $e = \text{else}$) becomes

$$S \rightarrow iEtSS' \mid a, \quad S' \rightarrow eS \mid \varepsilon, \quad E \rightarrow b,$$

which is exactly the slides’ homework grammar. Left factoring removes the common prefix, but this grammar is ambiguous, and §9 shows that it still has one conflict.

Warning / Gotcha: What the transformations do not do

- They preserve the *language*, not the *parse trees*: $E \rightarrow TE'$ builds a right-leaning tree for **a+b+c**. Later phases must still treat **+** as left-associative.
- They cannot remove ambiguity. An ambiguous grammar is never LL(1).
- Removing left recursion and left factoring do not guarantee LL(1). Always check with FIRST and FOLLOW (§9).
- The slides’ first example $A \rightarrow Abc \mid a$ is left-recursive; it illustrates what a top-down tree looks like, but a top-down parser could not use that grammar as written.

4 Recursive Descent Parsers

4.1 First, brute force

The simplest top-down parser tries the alternatives of a non-terminal in order and **backtracks** when one fails: it returns to the input position where the choice was made and tries the next alternative.

Worked example: Backtracking on cad

$S \rightarrow cAd$, $A \rightarrow ab \mid a$.

1. Expand $S \rightarrow cAd$; match **c**.
2. Try $A \rightarrow ab$: match **a**; expect **b** but see **d**. **Fail**: go back to just after **c**.
3. Try $A \rightarrow a$: match **a**. Then match **d** from S ’s production. End of input: **accept**.

Backtracking is simple but can take exponential time, and undoing work is awkward when the

parser also builds trees or runs semantic actions. Every parser in the rest of the chapter chooses correctly the first time, by looking at the next token.

4.2 The recursive descent parser

Concept: Recursive descent

A **recursive descent parser** is a top-down parser built from a set of mutually recursive procedures, **one per non-terminal**. The procedure for A chooses one of A 's productions by looking at the current token, the *lookahead*, and then works through the right side: it calls **match** for each terminal and the procedure of each non-terminal. The structure of the program mirrors the grammar.

Why “recursive descent”? *Descent*: the parse proceeds from the root of the tree down to the leaves, which is top-down. *Recursive*: the procedures call one another, and themselves, exactly as the grammar's non-terminals refer to one another. At any moment the program's call stack holds the path from the root of the parse tree to the node being built.

Worked example: The slides' parser for $E \rightarrow i E'$, $E' \rightarrow + i E' \mid \varepsilon$

```
E() {                                // E -> i E'
    if (look_ahead == 'i') {
        match('i');
        E'();
    }
}
E'() {                                // E' -> + i E'
    / eps
    if (look_ahead == '+') {
        match('+');
        match('i');
        E'();
    }
    else
        return;                      // the eps
    choice
}
```

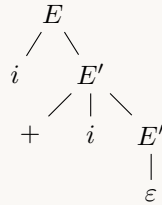
```
match(char c) {
    if (look_ahead == c)
        look_ahead = getchar();    // consume,
        read next
    else
        printf("ERROR!");
}
main() {
    E();
    if (look_ahead == '$')
        printf("Parsing Successful!");
}
```

How each piece maps to the grammar:

- **match(c)** consumes one terminal: it checks that the lookahead is c and reads the next character.
- **E'()** has two productions to choose from. It takes $E' \rightarrow +iE'$ when the lookahead is $+$, and the ε -production (do nothing, return) otherwise.
- **main** parses one E and then checks that the whole input was consumed: the next symbol must be the end marker $\$$.

Worked example: Trace on i+i\$

Call stack (top on the right)	Lookahead	Action
main	i	call E
main E	i	match('i'), call E'
main E E'	+	lookahead is +: match('+'), match('i'), call E'
main E E' E'	\$	lookahead is not +: return ($E' \rightarrow \varepsilon$)
main E E'	\$	return
main E	\$	return
main	\$	lookahead is \$: "Parsing Successful!"



The calls trace the parse tree depth first, left to right; each procedure call is one interior node.

Debugging: The slides' code, compiled and run

Compiled with the missing declarations added and nothing else changed, the parser was run on five inputs:

Input	Slides' code	With look_ahead = getchar() first	Correct behaviour
i+i\$	prints nothing	"Parsing Successful!"	accept
i+\$	prints nothing	"ERROR!Parsing Successful!"	reject
ii\$	prints nothing	prints nothing	reject
+i\$	prints nothing	prints nothing	reject

1. **The lookahead is never initialised.** main calls E() before reading any character, so look_ahead is 0, E does nothing, and even valid input produces no output. Fix: look_ahead = getchar(); at the start of main.
2. **An error does not stop the parse.** match prints "ERROR!" and carries on, so i+\$ prints "ERROR!Parsing Successful!". Fix: stop (or recover) on error.
3. **Silent failures.** E does nothing on a wrong first token, E' returns on *any* token other than +, and main prints nothing on failure. Fix: report an error in each case.

Method: A correct version, and where the decisions come from

```

void E(void) {
    /* E -> i E' */
    if (look_ahead == 'i') { match('i'); Ep(); }
    else error("expected 'i'");
}

void Ep(void) {
    /* E' -> + i E' | eps */
    if (look_ahead == '+') { match('+'); match('i'); Ep(); } /* '+' in FIRST(+ i E') */
    else if (look_ahead == '$') return; /* '$' in FOLLOW(E') */
    else error("expected '+' or end of input");
}
  
```

With main reading the first character and error stopping the program, this version accepts i+i\$ and i+i+i\$ and rejects the other three inputs with a precise message (checked).

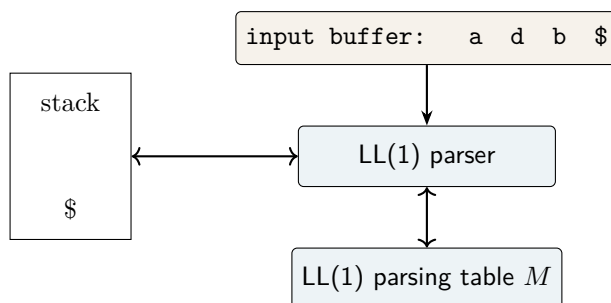
Look at the two tests in **Ep**. The production $E' \rightarrow +iE'$ is chosen when the lookahead can *start* it: that is $\text{FIRST}(+iE') = \{+\}$. The ε -production is chosen when the lookahead can *come after* E' : that is $\text{FOLLOW}(E') = \{\$\}$. Any other token is an error. These two sets are exactly what the rest of the chapter computes, and an LL(1) table is nothing but these tests written down for every non-terminal.

5 The LL(1) Parser

LL(1) reads as three choices:

- the first **L**: the input is scanned from **L**eft to right;
- the second **L**: the parser builds a **L**eftmost derivation (it is top-down);
- the **1**: it looks at **one** token of lookahead to choose each production.

$\text{LL}(k)$ uses k tokens of lookahead.



The **stack** holds the part of the sentential form still to be matched, with \$ at the bottom. The **table** $M[A, a]$ says which production to use when A is on top of the stack and a is the lookahead. Building the table needs two sets for every non-terminal: **FIRST** and **FOLLOW**.

6 FIRST Sets

Definition 6.1 (FIRST). For a string of grammar symbols α ,

$$\text{FIRST}(\alpha) = \{a \in T : \alpha \Rightarrow^* a\beta \text{ for some } \beta\} \cup \{\varepsilon : \alpha \Rightarrow^* \varepsilon\}.$$

In words (the slides' version): derive every possible string from α , and collect the terminals that can appear *first*. If α can vanish entirely, add ε .

Nullable symbols. A non-terminal A is **nullable** if $A \Rightarrow^* \varepsilon$. Compute the set of nullable symbols first, by iteration: mark A nullable if it has a production $A \rightarrow \varepsilon$, or a production $A \rightarrow Y_1 \cdots Y_k$ whose symbols are all already marked nullable; repeat until nothing changes. A terminal is never nullable.

Method: Computing FIRST

1. If X is a terminal, $\text{FIRST}(X) = \{X\}$.
2. If $X \rightarrow \varepsilon$ is a production, add ε to $\text{FIRST}(X)$.
3. For a production $X \rightarrow Y_1 Y_2 \cdots Y_k$:
 - add $\text{FIRST}(Y_1) \setminus \{\varepsilon\}$;

- if Y_1 is nullable, also add $\text{FIRST}(Y_2) \setminus \{\varepsilon\}$; if Y_1, Y_2 are both nullable, also add $\text{FIRST}(Y_3) \setminus \{\varepsilon\}$; and so on;
 - if *all* of $Y_1, \dots, Y_k$ are nullable, add ε .
4. Repeat step 3 over all productions until no set changes (a *fixed point*). Sets only grow and are bounded, so this stops.
- The same rule 3 gives FIRST of any string $\alpha = Y_1 \cdots Y_k$, and $\text{FIRST}(\varepsilon) = \{\varepsilon\}$.

Key insight. The scan through $Y_1, Y_2, \dots$ stops at the first symbol that is *not* nullable. Everything after it can never be first. The most common mistake is to stop too early (ignoring that Y_1 can vanish) or to carry on too far (adding $\text{FIRST}(Y_3)$ when Y_2 is not nullable).

Worked example: The slides' two warm-ups

- $S \rightarrow aABC$, $A \rightarrow b$, $B \rightarrow c$, $C \rightarrow d$: every right side starts with a terminal, so $\text{FIRST}(S) = \{a\}$, $\text{FIRST}(A) = \{b\}$, $\text{FIRST}(B) = \{c\}$, $\text{FIRST}(C) = \{d\}$.
- $S \rightarrow ABC$, $A \rightarrow a \mid \varepsilon$, $B \rightarrow b$, $C \rightarrow c$: A is nullable, B is not. $\text{FIRST}(S) = \text{FIRST}(A) \setminus \{\varepsilon\} \cup \text{FIRST}(B) = \{a, b\}$. The scan stops at B , so $c \notin \text{FIRST}(S)$, and $\varepsilon \notin \text{FIRST}(S)$ because B cannot vanish.

Worked example: The expression grammar, in dependency order

$$E \rightarrow TE', \quad E' \rightarrow +TE' \mid \varepsilon, \quad T \rightarrow FT', \quad T' \rightarrow *FT' \mid \varepsilon, \quad F \rightarrow \text{id} \mid (E).$$

Nullable: E' and T' (they have ε -productions); E, T, F are not. Work from the bottom of the dependency chain:

Set	Reason	Result
$\text{FIRST}(F)$	both productions start with a terminal	$\{\text{id}, (\}$
$\text{FIRST}(T')$	$*FT'$ starts with $*$; plus ε	$\{*, \varepsilon\}$
$\text{FIRST}(T)$	$T \rightarrow FT'$, and F is not nullable, so stop at F	$\text{FIRST}(F) = \{\text{id}, (\}$
$\text{FIRST}(E')$	$+TE'$ starts with $+$; plus ε	$\{+, \varepsilon\}$
$\text{FIRST}(E)$	$E \rightarrow TE'$, stop at T	$\text{FIRST}(T) = \{\text{id}, (\}$

These are the slides' values.

7 FOLLOW Sets

Definition 7.1 (FOLLOW). For a non-terminal A ,

$$\text{FOLLOW}(A) = \{ a \in T : S \$ \Rightarrow^* \alpha A a \beta \},$$

the terminals that can appear *immediately to the right of* A in some sentential form, with $\$$ included when A can be the last symbol. FOLLOW sets never contain ε : they are sets of tokens that can actually be seen next.

Method: Computing FOLLOW

1. Put $\$$ in $\text{FOLLOW}(S)$ for the start symbol S .
2. For every production $A \rightarrow \alpha B \beta$ (an occurrence of the non-terminal B with β after it): add $\text{FIRST}(\beta) \setminus \{\varepsilon\}$ to $\text{FOLLOW}(B)$.
3. If moreover β is nullable (including $\beta = \varepsilon$, i.e. B is last): add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.

4. Repeat over every occurrence of every non-terminal until no set changes.

The slides' three rules are these, read on examples:

- “the following terminal is selected”: rule 2 when β starts with a terminal;
- “the FIRST of the following non-terminal is selected”: rule 2 when β starts with a non-terminal, and if that FIRST contains ε , keep going to the next symbol, and finally to rule 3;
- “if it is the rightmost in the right side, FOLLOW of the left side is selected”: rule 3.

Key insight. Rule 3 is the one that moves information *downward*: whatever can follow A can follow the last (non-vanishing-suffix) symbol of A 's right side, because that symbol sits at the right edge of A 's subtree.

Worked example: The slides' warm-up: $S \rightarrow ABC$, $A \rightarrow a$, $B \rightarrow b \mid \varepsilon$, $C \rightarrow c$

$\text{FOLLOW}(S) = \{\$ \}$ (rule 1). A is followed by BC : $\text{FIRST}(BC) = \{b\} \cup \{c\}$ because B is nullable, so $\text{FOLLOW}(A) = \{b, c\}$. B is followed by C : $\text{FOLLOW}(B) = \{c\}$. C is last in S 's production: $\text{FOLLOW}(C) = \text{FOLLOW}(S) = \{\$ \}$.

Worked example: The expression grammar, step by step

- $\text{FOLLOW}(E)$: $\$$ by rule 1; $)$ because $F \rightarrow (E)$ puts $)$ after E . Result $\{\$,)\}$.
- $\text{FOLLOW}(E')$: E' is last in $E \rightarrow TE'$, so it gets $\text{FOLLOW}(E)$; in $E' \rightarrow +TE'$ it gets its own set (nothing new). Result $\{\$,)\}$.
- $\text{FOLLOW}(T)$: T is followed by E' in both $E \rightarrow TE'$ and $E' \rightarrow +TE'$. Rule 2 gives $\text{FIRST}(E') \setminus \{\varepsilon\} = \{+\}$. Since E' is nullable, rule 3 adds $\text{FOLLOW}(E)$ and $\text{FOLLOW}(E')$, both $\{\$,)\}$. Result $\{+, \$,)\}$. (The slide writes $\text{FOLLOW}(T) = \text{FIRST}(E') = \{+, \varepsilon\}$ and circles the ε : that circled ε is exactly “replace by FOLLOW of the left side”.)
- $\text{FOLLOW}(T')$: last in $T \rightarrow FT'$, so it gets $\text{FOLLOW}(T) = \{+, \$,)\}$.
- $\text{FOLLOW}(F)$: followed by T' , so $\{*\}$, and T' is nullable, so add $\text{FOLLOW}(T)$ and $\text{FOLLOW}(T')$. Result $\{*, +, \$,)\}$.

Production	FIRST of the left side	FOLLOW of the left side
$E \rightarrow TE'$	$\{\text{id}, ()$	$\{\$,)\}$
$E' \rightarrow +TE' \mid \varepsilon$	$\{+, \varepsilon\}$	$\{\$,)\}$
$T \rightarrow FT'$	$\{\text{id}, ()$	$\{+, \$,)\}$
$T' \rightarrow *FT' \mid \varepsilon$	$\{*, \varepsilon\}$	$\{+, \$,)\}$
$F \rightarrow \text{id} \mid (E)$	$\{\text{id}, ()$	$\{*, +, \$,)\}$

7.1 Cycles, and why iteration is needed

Rule 3 creates inclusions $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$, and these can form cycles. Then no order of computation finishes in one sweep “by hand logic”; the algorithm simply repeats until nothing changes.

Worked example: A FOLLOW cycle

$S \rightarrow Ax \mid By$, $A \rightarrow aB$, $B \rightarrow bA \mid \varepsilon$.

- Rule 2 gives $x \in \text{FOLLOW}(A)$ (from $S \rightarrow Ax$) and $y \in \text{FOLLOW}(B)$ (from $S \rightarrow By$).
- Rule 3 gives two inclusions: $A \rightarrow aB$ puts B last, so $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$; $B \rightarrow bA$ puts A last, so $\text{FOLLOW}(B) \subseteq \text{FOLLOW}(A)$.

Draw a node per non-terminal and an edge $A \rightarrow B$ for each inclusion $\text{FOLLOW}(A) \subseteq$

$\text{FOLLOW}(B)$: here $A \rightarrow B \rightarrow A$ is a **cycle**. Every set on a cycle includes every other, so they are *equal*, whatever the order of computation. Hence $\text{FOLLOW}(A) = \text{FOLLOW}(B) = \{x, y\}$, and $\text{FOLLOW}(S) = \{\$ \}$.

Iterating (productions top to bottom, updating in place): pass 1 gives $\text{FOLLOW}(A) = \{x\}$, then $\text{FOLLOW}(B) = \{y\}$, then $\text{FOLLOW}(B) = \{x, y\}$ from $A \rightarrow aB$, then $\text{FOLLOW}(A) = \{x, y\}$ from $B \rightarrow bA$. Pass 2 changes nothing, so the algorithm stops. With a different order of productions, more passes can be needed; the final sets are always the same.

Key insight. The fixed point is unique: the algorithm computes the *smallest* sets that satisfy all the rules, and that does not depend on the order. The order only changes how many passes it takes.

7.2 The slides' exercises, solved

Worked example: GATE 2017: $P \rightarrow xQRS, Q \rightarrow yz \mid z, R \rightarrow w \mid \varepsilon, S \rightarrow y$. What is $\text{FOLLOW}(Q)$?

Q is followed by RS . $\text{FIRST}(R) = \{w, \varepsilon\}$ gives w ; R is nullable, so continue to S : $\text{FIRST}(S) = \{y\}$; S is not nullable, so stop. $\text{FOLLOW}(Q) = \{w, y\}$: **answer (C)**. Option (D) $\{w, \varepsilon\}$ is wrong twice: ε is never in a FOLLOW set, and it misses y . Option (A) $\{R\}$ is not even a set of terminals.

Worked example: $S \rightarrow aBDh, B \rightarrow cC, C \rightarrow bC \mid \varepsilon, D \rightarrow EF, E \rightarrow g \mid \varepsilon, F \rightarrow f \mid \varepsilon$

Nullable: C, E, F , and D (both E and F are). So $\text{FIRST}(D) = \{g, f, \varepsilon\}$.

- $\text{FOLLOW}(B)$: B is followed by Dh ; $\text{FIRST}(Dh) = \{g, f\} \cup \{h\}$ since D is nullable. So $\{g, f, h\}$.
- $\text{FOLLOW}(C)$: C is last in $B \rightarrow cC$ (and in $C \rightarrow bC$), so $\text{FOLLOW}(C) = \text{FOLLOW}(B) = \{g, f, h\}$.
- $\text{FOLLOW}(D) = \{h\}$. $\text{FOLLOW}(E)$: E is followed by F , giving f , and F is nullable, so add $\text{FOLLOW}(D)$: $\{f, h\}$. $\text{FOLLOW}(F) = \text{FOLLOW}(D) = \{h\}$.

Grammar	FIRST	FOLLOW
$S \rightarrow ABCDE, A \rightarrow a \mid \varepsilon,$ $B \rightarrow b \mid \varepsilon, C \rightarrow c, D \rightarrow d \mid \varepsilon,$ $E \rightarrow e \mid \varepsilon$	$S\{a, b, c\}, A\{a, \varepsilon\}, B\{b, \varepsilon\},$ $C\{c\}, D\{d, \varepsilon\}, E\{e, \varepsilon\}$	$S\{\$, A\{b, c\}, B\{c\}, C\{d, e, \$\},$ $D\{e, \$\}, E\{\$ \}$
$S \rightarrow Bb \mid Cd, B \rightarrow aB \mid \varepsilon,$ $C \rightarrow cC \mid \varepsilon$	$S\{a, b, c, d\}, B\{a, \varepsilon\}, C\{c, \varepsilon\}$	$S\{\$, B\{b\}, C\{d\}$
$S \rightarrow ACB \mid CbB \mid Ba,$ $A \rightarrow da \mid BC, B \rightarrow g \mid \varepsilon,$ $C \rightarrow h \mid \varepsilon$	$S\{d, g, h, b, a, \varepsilon\}, A\{d, g, h, \varepsilon\},$ $B\{g, \varepsilon\}, C\{h, \varepsilon\}$	$S\{\$, A\{h, g, \$\}, B\{\$, a, h, g\},$ $C\{g, \$, b, h\}$

All three agree with the slides and were checked by program. Two details to notice: in the first grammar C is not nullable, so $\text{FIRST}(S)$ stops at c and does not contain ε . In the third, S is nullable (through $S \rightarrow ACB$ with all three nullable), so $\varepsilon \in \text{FIRST}(S)$.

8 The LL(1) Parsing Table and the Stack Algorithm

Method: Building the table M

Rows: non-terminals. Columns: terminals and \$. For each production $A \rightarrow \alpha$:

1. for each terminal $a \in \text{FIRST}(\alpha)$, put $A \rightarrow \alpha$ in $M[A, a]$;
 2. if α is nullable ($\varepsilon \in \text{FIRST}(\alpha)$), put $A \rightarrow \alpha$ in $M[A, b]$ for each $b \in \text{FOLLOW}(A)$, including \$.
- Empty cells mean “syntax error”. If some cell receives two productions, the grammar is **not LL(1)**.

The slides phrase this as “ ε -productions are placed under FOLLOW, the remaining productions under FIRST”. Rule 2 is wider than that: it applies to every production whose right side *can derive* ε , not only to $A \rightarrow \varepsilon$. The slides’ own table for $S \rightarrow AB$ (A, B nullable) places $S \rightarrow AB$ under \$, which only rule 2 explains.

Key insight. Why the rules are right. If the lookahead is a and $a \in \text{FIRST}(\alpha)$, then α can start with a , so $A \rightarrow \alpha$ is a candidate. If α can vanish and a can follow A , then “ A produces nothing here and a belongs to whatever comes next” is a candidate. No other production can lead to a successful parse with a next. LL(1) means there is never more than one candidate.

Worked example: The table for the expression grammar

	id	(	)	*	+	\$
E	$E \rightarrow TE'$	$E \rightarrow TE'$				
E'			$E' \rightarrow \varepsilon$		$E' \rightarrow +TE'$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$	$T \rightarrow FT'$				
T'			$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow \text{id}$	$F \rightarrow (E)$				

$E' \rightarrow \varepsilon$ goes under $\text{FOLLOW}(E') = \{\}, \{\$, \}$, and $T' \rightarrow \varepsilon$ under $\text{FOLLOW}(T') = \{+, \), \{\$, \}$. No cell holds two productions: the grammar is LL(1).

Theorem 8.1 (The LL(1) condition). *A grammar (with every non-terminal reachable and productive) is LL(1) if and only if, for every non-terminal A and every pair of distinct productions $A \rightarrow \alpha \mid \beta$:*

1. **no FIRST/FIRST clash:** $\text{FIRST}(\alpha)$ and $\text{FIRST}(\beta)$ share no terminal;
2. *at most one of α, β is nullable;*
3. **no FIRST/FOLLOW clash:** *if β is nullable, then $\text{FIRST}(\alpha)$ and $\text{FOLLOW}(A)$ share no terminal.*

Proof. By the construction, $A \rightarrow \alpha$ lands in $M[A, a]$ exactly when $a \in \text{FIRST}(\alpha)$, or α is nullable and $a \in \text{FOLLOW}(A)$. Two productions α, β of A share a cell $M[A, a]$ in one of three ways:

- a is in both FIRST sets, which violates condition 1;
- both are nullable and $a \in \text{FOLLOW}(A)$, which violates condition 2 (such an a exists, since a reachable A has a nonempty FOLLOW set);
- one is nullable, say β , $a \in \text{FOLLOW}(A)$, and $a \in \text{FIRST}(\alpha)$, which violates condition 3.

So the table has no doubly-filled cell exactly when the three conditions hold. □

Method: The LL(1) parsing algorithm

```

1 push $ then  $S$  on the stack; let  $a$  be the first token;
2 while true do
3    $X \leftarrow$  top of the stack;
4   if  $X = \$$  and  $a = \$$  then
5     accept
6   else if  $X$  is a terminal then
7     if  $X = a$  then pop  $X$ ;  $a \leftarrow$  next token (match);
8     else error;
9   else if  $M[X, a] = X \rightarrow Y_1 \cdots Y_k$  then
10    pop  $X$ ; push  $Y_k, \dots, Y_1$  (so  $Y_1$  is on top)
11  else error (empty cell);

```

The right side is pushed *in reverse* so that its first symbol is matched first. Popping X and pushing nothing implements $X \rightarrow \varepsilon$.

Worked example: The slides' trace: $S \rightarrow aABb$, $A \rightarrow c \mid \varepsilon$, $B \rightarrow d \mid \varepsilon$, input adb

FIRST: $S\{a\}$, $A\{c, \varepsilon\}$, $B\{d, \varepsilon\}$. FOLLOW: $S\{\$\}$, $A\{d, b\}$ (A is followed by Bb , B nullable), $B\{b\}$.

					Stack (top right)	Input	Action
					$\$ S$	$a \ d \ b \ \$$	$S \rightarrow aABb$
					$\$ bBAa$	$a \ d \ b \ \$$	match a
S	$S \rightarrow aABb$				$\$ bBA$	$d \ b \ \$$	$A \rightarrow \varepsilon$
A		$A \rightarrow \varepsilon$	$A \rightarrow c$	$A \rightarrow \varepsilon$	$\$ bB$	$d \ b \ \$$	$B \rightarrow d$
B		$B \rightarrow \varepsilon$		$B \rightarrow d$	$\$ bd$	$d \ b \ \$$	match d
					$\$ b$	$b \ \$$	match b
					$\$$	$\$$	accept

At the third line the parser must decide whether A produces c or nothing; the lookahead d is in FOLLOW(A), not in FIRST(c), so A vanishes.

Worked example: $id + id * id$ with the expression grammar, and an error

Stack	Input	Action	Stack	Input	Action
$\$ E$	$id+id*id\$$	$E \rightarrow TE'$	$\$ E'T$	$id*id\$$	$T \rightarrow FT'$
$\$ E'T$	$id+id*id\$$	$T \rightarrow FT'$	$\$ E'T'F$	$id*id\$$	$F \rightarrow id$
$\$ E'T'F$	$id+id*id\$$	$F \rightarrow id$	$\$ E'T'id$	$id*id\$$	match
$\$ E'T'id$	$id+id*id\$$	match	$\$ E'T'$	$*id\$$	$T' \rightarrow *FT'$
$\$ E'T'$	$+id*id\$$	$T' \rightarrow \varepsilon$	$\$ E'T'F*$	$*id\$$	match
$\$ E'$	$+id*id\$$	$E' \rightarrow +TE'$	$\$ E'T'F$	$id\$$	$F \rightarrow id$, match
$\$ E'T+$	$+id*id\$$	match	$\$ E'T'$ then $\$ E'$	$\$$	$T' \rightarrow \varepsilon$, $E' \rightarrow \varepsilon$
			$\$$	$\$$	accept

On the wrong input $id + * id$, the parser reaches stack $\$ E'T$ with lookahead $*$. The cell $M[T, *]$ is empty: a T can never start with $*$. The error is reported *at the first token that cannot fit*, which is the strength of predictive parsing (checked by program).

9 Is This Grammar LL(1)?

Method: Deciding LL(1) quickly

1. Remove nothing; take the grammar as given. Compute nullable, FIRST and FOLLOW.
2. For every non-terminal with two or more productions, check the three conditions of Theorem 8.1: FIRST sets of the alternatives disjoint; at most one nullable alternative; for the nullable one, the *other* alternatives' FIRST sets disjoint from FOLLOW(A).
3. Non-terminals with a single production can never cause a conflict.
4. If a check fails, name the non-terminal, the two productions and the terminal: that is the conflicting cell.

Grammar	FIRST / FOLLOW	Verdict and reason
$S \rightarrow aSbS \mid bSaS \mid \varepsilon$	$S: \{a, b, \varepsilon\} / \{a, b, \$\}$	Not LL(1): $S \rightarrow \varepsilon$ goes under FOLLOW $\ni a, b$, colliding with $S \rightarrow aSbS$ on a and $S \rightarrow bSaS$ on b (FIRST/FOLLOW).
$S \rightarrow (S) \mid \varepsilon$	$S: \{(\varepsilon) / \{), \$\}$	LL(1): ($\notin$ FOLLOW(S)).
$S \rightarrow AaAb \mid BbBa, A \rightarrow \varepsilon, B \rightarrow \varepsilon$	$S: \{a, b\}; A, B: \{\varepsilon\} / \{a, b\}$	LL(1): the two alternatives of S start with a and b .
$S \rightarrow A \mid a, A \rightarrow a$	$S, A: \{a\} / \{\$ \}$	Not LL(1): FIRST/FIRST on a (the grammar is also ambiguous).
$S \rightarrow aB \mid \varepsilon, B \rightarrow bC \mid \varepsilon, C \rightarrow cS \mid \varepsilon$	FOLLOW of all three is $\{\$ \}$	LL(1): each ε goes under $\$$ only.
$S \rightarrow AB, A \rightarrow a \mid \varepsilon, B \rightarrow b \mid \varepsilon$	$A: \{a, \varepsilon\} / \{b, \$\}; B: \{b, \varepsilon\} / \{\$ \}$	LL(1): $a \notin$ FOLLOW(A), $b \notin$ FOLLOW(B).
$S \rightarrow aSA \mid \varepsilon, A \rightarrow c \mid \varepsilon$	$A: \{c, \varepsilon\} / \{c, \$ \}$	Not LL(1): $c \in$ FOLLOW(A), so $A \rightarrow c$ and $A \rightarrow \varepsilon$ collide on c .
$S \rightarrow A, A \rightarrow Bb \mid Cd, B \rightarrow aB \mid \varepsilon, C \rightarrow cC \mid \varepsilon$	$A: \{a, b, c, d\}; B / \{b\}; C / \{d\}$	LL(1): FIRST(Bb) = $\{a, b\}$ and FIRST(Cd) = $\{c, d\}$ are disjoint.
$S \rightarrow aAa \mid \varepsilon, A \rightarrow abS \mid \varepsilon$	$S: \{a, \varepsilon\} / \{a, \$ \}; A: \{a, \varepsilon\} / \{a\}$	Not LL(1): conflicts on a for both S and A .

All verdicts agree with the slides and were checked by building the tables.

Worked example: Homework: $S \rightarrow iEtSS' \mid a, S' \rightarrow eS \mid \varepsilon, E \rightarrow b$

FIRST. FIRST(S) = $\{i, a\}$, FIRST(S') = $\{e, \varepsilon\}$, FIRST(E) = $\{b\}$.

FOLLOW.

- FOLLOW(S) contains $\$$. In $S \rightarrow iEtSS'$, the inner S is followed by S' : add FIRST(S') $\setminus \{\varepsilon\} = \{e\}$, and since S' is nullable, add FOLLOW(S).
- S' is last in $S \rightarrow iEtSS'$, so FOLLOW(S) $\subseteq$ FOLLOW(S'); S is last in $S' \rightarrow eS$, so FOLLOW(S') $\subseteq$ FOLLOW(S). A cycle: they are equal, $\{e, \$\}$.
- E is followed by t : FOLLOW(E) = $\{t\}$.

Table. $S' \rightarrow eS$ goes under e ; $S' \rightarrow \varepsilon$ goes under FOLLOW(S') = $\{e, \$\}$. The cell $M[S', e]$ holds both: **not LL(1)**, a FIRST/FOLLOW conflict.

	a	b	e	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow eS, S' \rightarrow \varepsilon$			$S' \rightarrow \varepsilon$
E		$E \rightarrow b$				

Meaning. This is the dangling else. In `if b then if b then a else a`, the `else` can belong to either `if`: the grammar is ambiguous, and no grammar transformation alone will make it LL(1). Real parsers resolve the conflict by always choosing $S' \rightarrow eS$ on e : the `else` attaches to the nearest unmatched `if`, which is the rule of C, Java and Pascal. With that choice the table is deterministic.

Worked example: A bonus from the FIRST/FOLLOW exercises

The slides compute FIRST and FOLLOW for $S \rightarrow ACB \mid CbB \mid Ba$, $A \rightarrow da \mid BC$, $B \rightarrow g \mid \varepsilon$, $C \rightarrow h \mid \varepsilon$ without asking whether it is LL(1). It is not: for instance $g \in \text{FOLLOW}(B)$, so $B \rightarrow g$ and $B \rightarrow \varepsilon$ collide on g , and $S \rightarrow ACB$ and $S \rightarrow Ba$ both start with g .

10 Tricks, Techniques and Classic Problems

Technique: Exam-speed recipes

- **Nullable first.** List the nullable non-terminals before anything else; every FIRST and FOLLOW computation depends on them.
- **FIRST of $Y_1Y_2\cdots$:** take FIRST of each symbol, without ε , *until* you meet one that is not nullable. Add ε only if all of them are nullable.
- **FOLLOW:** go through every *occurrence* of B on a right side. Look right: add FIRST of what follows. If what follows can vanish (or nothing follows), add FOLLOW of the left side.
- **Inclusion graph:** when FOLLOW sets refer to each other, draw edges $A \rightarrow B$ for $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$. Sets on a cycle are equal.
- **Table:** each production under FIRST of its right side; if the right side can vanish, also under FOLLOW of its left side.
- **LL(1) check:** only non-terminals with several productions matter; name the conflicting cell.

Worked example: Classic short questions

- P1. Can a left-recursive grammar be LL(1)?** No, by Proposition 3.1 (as long as the non-recursive alternative can start with a terminal).
- P2. Can an ambiguous grammar be LL(1)?** No. Two different leftmost derivations of the same string first differ at a step where the same non-terminal A is expanded by two different productions, with the same remaining input. Both productions must then be valid choices for that lookahead, so both sit in the same cell.
- P3. Can ε be in a FOLLOW set?** Never. FOLLOW contains tokens that can be seen next, and $\$$ plays the role of “nothing left”.
- P4. Is $\text{FOLLOW}(S) = \{\$\}$ always?** No. $\$$ is always in it, but if S appears on a right side it can get more: $S \rightarrow (S) \mid \varepsilon$ has $\text{FOLLOW}(S) = \{), \$\}$.
- P5. If $\text{FIRST}(A) = \text{FIRST}(B)$, is $\text{FOLLOW}(A) = \text{FOLLOW}(B)$?** No. FOLLOW depends on where a non-terminal is *used*, FIRST on what it *produces*. In $S \rightarrow Ax \mid By$, $A \rightarrow a$, $B \rightarrow a$, the FIRST sets are both $\{a\}$ but the FOLLOW sets are $\{x\}$ and $\{y\}$.
- P6. Do FIRST/FIRST-disjoint alternatives guarantee LL(1)?** No: condition 3 (FIRST/-FOLLOW) can still fail, as in $S \rightarrow aSA \mid \varepsilon$, $A \rightarrow c \mid \varepsilon$.
- P7. Can FIRST and FOLLOW be computed for a left-recursive grammar?** Yes. The definitions and the fixed-point algorithms work for any context-free grammar; the grammar just will not be LL(1).
- P8. What does an empty table cell mean at run time?** A syntax error detected at the

current token: no sentence of the language can continue this way.

Warning / Gotcha: Gotchas

- Putting ε in a FOLLOW set, or writing ε as a table column.
- Forgetting $\$$ in $\text{FOLLOW}(S)$, or forgetting to propagate it through rule 3.
- Stopping the FIRST scan at a nullable symbol, or not stopping at a non-nullable one.
- Using only the *first* occurrence of a non-terminal when computing its FOLLOW; every occurrence counts.
- Placing only literal $A \rightarrow \varepsilon$ under FOLLOW: every production whose right side is nullable goes there.
- Computing FOLLOW in one sweep when the rules form a cycle; iterate until nothing changes.
- Pushing a right side onto the stack in the wrong order: push it reversed so its first symbol is on top.

Map to the FIRST & FOLLOW Advanced Practice Sheet

Exercise	Prepared in
1–2: nullable symbols, long-range propagation, FIRST of strings	nullable symbols and the FIRST method (§6); FOLLOW rule 3 and when $\$$ propagates (§7)
3: fixed-point iteration, cyclic dependencies	the cycle example and the inclusion graph (§7)
4: eliminating indirect left recursion	the indirect algorithm and its worked example; Proposition 3.1; Theorem 8.1 (§3, §8)
5: left factoring a statement grammar	left factoring and the dangling else (§3, §9)
6: marking a student’s table	the gotchas above
7: true or false	classic questions P2–P7
8: table construction, conflict diagnosis	the table method, Theorem 8.1, the LL(1) check (§8, §9)
9: designing a grammar from its sets	run the FOLLOW rules backwards: to put b in $\text{FOLLOW}(X)$, write X just before b in some production, or last in a production of a non-terminal whose FOLLOW contains b

Notes on the original slides

- **Recursive descent code.** `main` never reads the first character, so the parser prints nothing even on valid input; `match` prints “ERROR!” but does not stop; failures produce no message. All three were confirmed by compiling and running the code (§4).
- **The first example grammar** $S \rightarrow aABe$, $A \rightarrow Abc \mid a$ is left-recursive. It illustrates the shape of a top-down tree, but a top-down parser could not use it as written.
- **FOLLOW rules.** The three rules on the slides omit the case where the following non-terminal is nullable; the slides apply it in the $\text{FOLLOW}(T)$ example by circling the ε . Rule 3 of §7 states it in general.
- **Table rule.** “ ε -productions are placed under FOLLOW” should read “productions whose right side can derive ε ”. The slides’ own table places $S \rightarrow AB$ under $\$$.
- **Non-determinism** on slide 6 means alternatives with a common prefix. Ambiguity is a separate obstacle that left factoring cannot remove (the homework grammar).

Summary: Chapter 5 in eight lines

1. A parser builds a parse tree from tokens. Top-down builds a leftmost derivation and decides *which production*; bottom-up builds a rightmost derivation in reverse and decides *when to reduce*.
2. Top-down parsing needs a grammar without left recursion (remove it: $A \rightarrow A\alpha \mid \beta$ becomes $A \rightarrow \beta A'$, $A' \rightarrow \alpha A' \mid \varepsilon$) and without common prefixes (left factor).
3. A recursive descent parser has one procedure per non-terminal and chooses a production from the lookahead.
4. $\text{FIRST}(\alpha)$: terminals that can start α , plus ε if α can vanish. Scan symbols until one is not nullable.
5. $\text{FOLLOW}(A)$: terminals that can come right after A , with $\$$ for the start symbol; never ε . Rule 3 copies FOLLOW of the left side when the rest can vanish. Iterate to a fixed point.
6. Table: $A \rightarrow \alpha$ under $\text{FIRST}(\alpha)$, and under $\text{FOLLOW}(A)$ if α is nullable. LL(1) means no cell holds two productions.
7. $\text{LL}(1) \iff$ no FIRST/FIRST clash, at most one nullable alternative, no FIRST/FOLLOW clash. Left-recursive and ambiguous grammars are never LL(1).
8. The stack algorithm expands by the table and matches terminals; an empty cell is a syntax error found at the first impossible token.